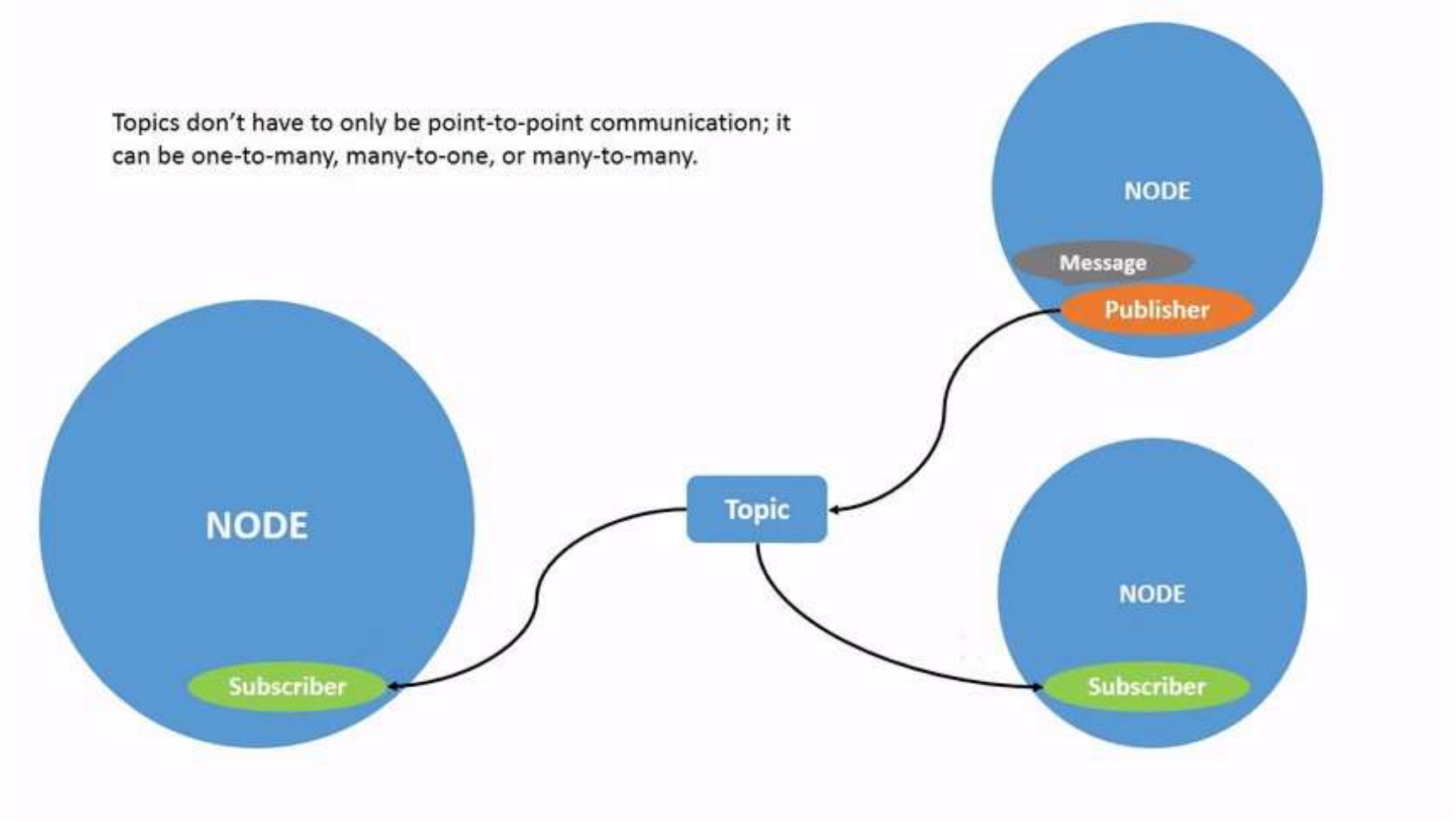
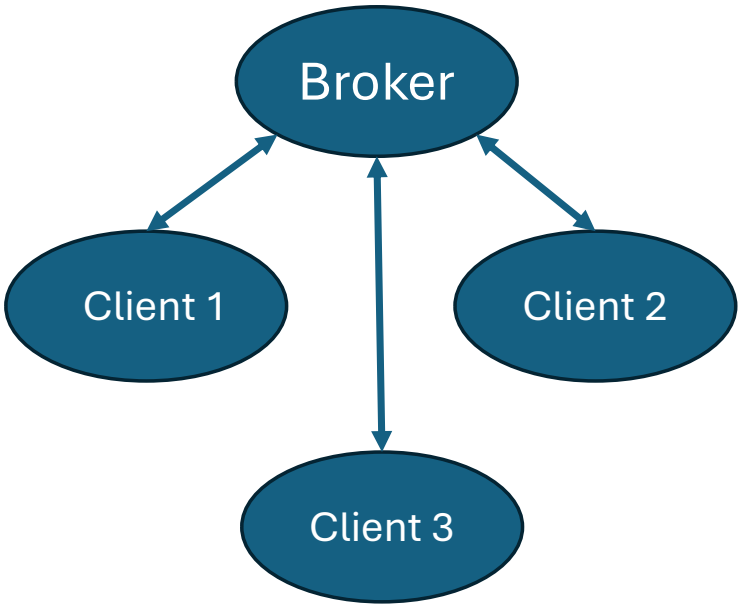
Tangent: ROS

ROS (zoom in)


Topics don't have to only be point-to-point communication; it can be one-to-many, many-to-one, or many-to-many.
NODE
Message
Publisher
NODE
Subscriber
Topic
NODE
Subscriber

MQTT vs ROS vs ROS2

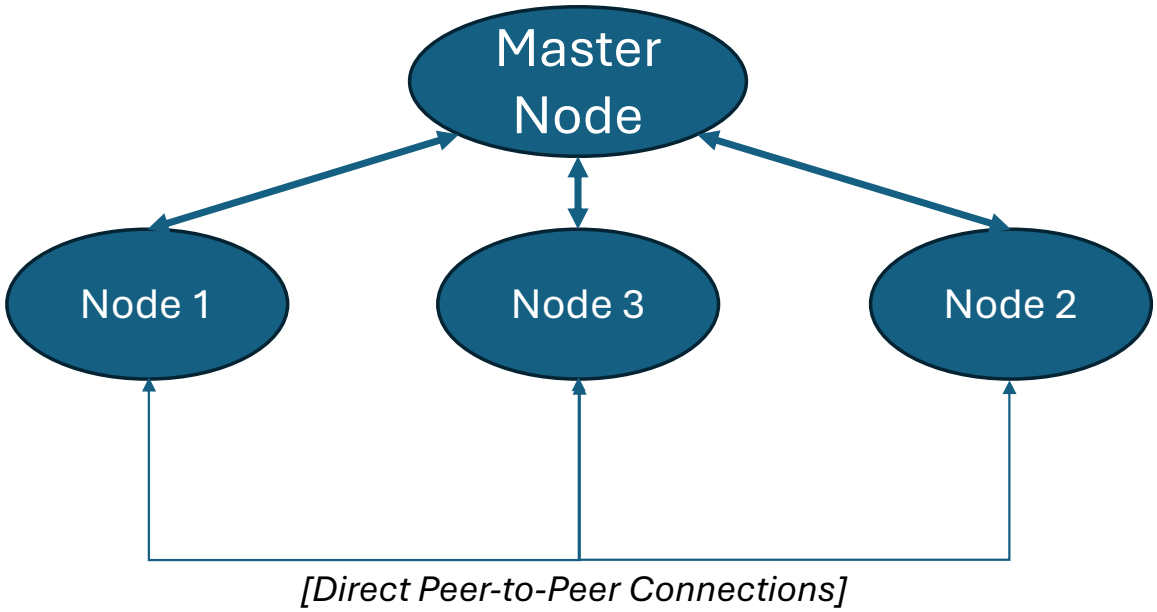
MQTT



All communication passes through the central Broker.

MQTT vs ROS vs ROS2

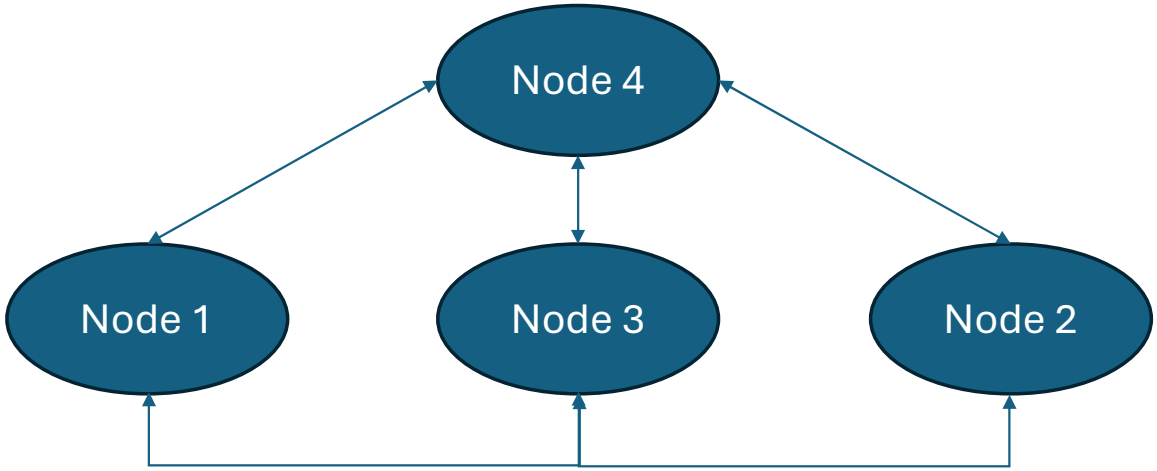
ROS



Nodes register with the Master Node.
After discovery, nodes establish direct connections for data exchange.

MQTT vs ROS vs ROS2

ROS2

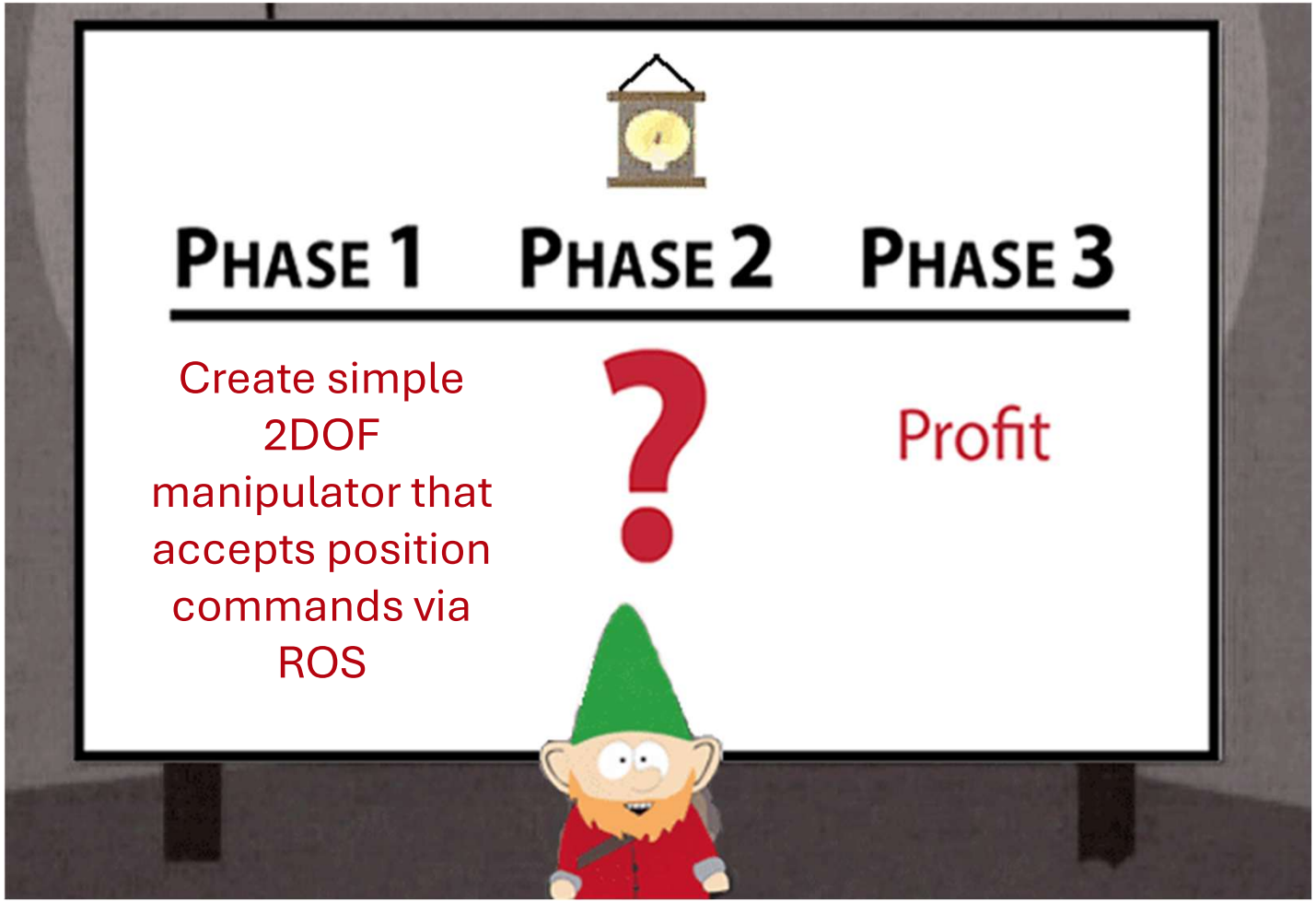


Nodes automatically discover each other using DDS (Data Distribution Service).
Communication is peer-to-peer without any central coordinator.

Main Differences:

- **MQTT:**
  - **Discovery & Communication:** Centralized via Broker.
  - **Architecture:** Clients communicate through the Broker only.
- **ROS (Version 1):**
  - **Discovery:** Centralized via Master Node.
  - **Communication:** Direct peer-to-peer between nodes.
- **ROS2:**
  - **Discovery & Communication:** Decentralized using DDS.
  - **Architecture:** Nodes discover and communicate directly.

Let’s put this in the simulator, shall we?



Minimal ROS2 2DOF simulator

- Create folder in /src in your MAS workspace -> e.g. “two_dof_arm”
- 3 folders -> “config”, “launch” , “urdf”
- 2 files -> package.xml, CMakeLists.txt

CMakeList.txt

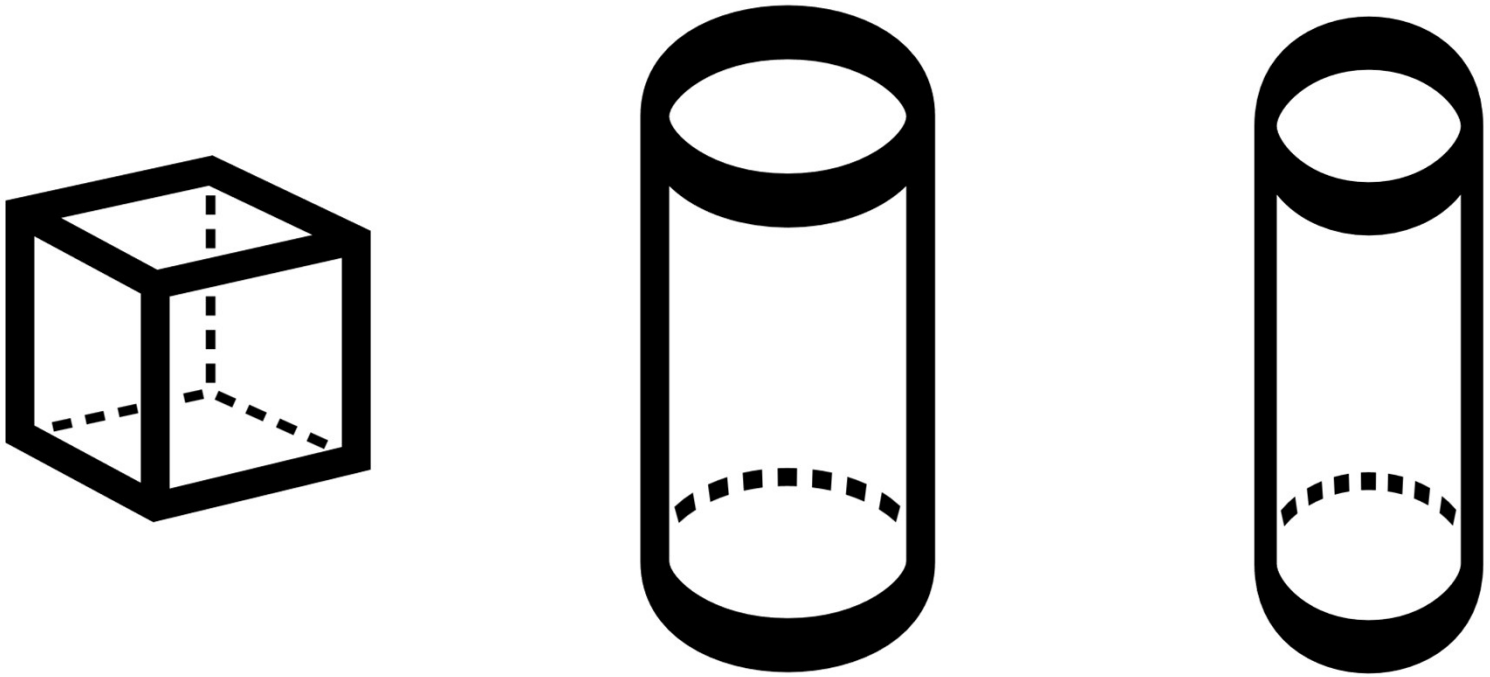
```
cmake_minimum_required(VERSION 3.8)
project(two_dof_arm)

find_package(ament_cmake REQUIRED)

install(DIRECTORY urdf config launch
  DESTINATION share/${PROJECT_NAME}
)

ament_package()
```

package.xml

```xml
<?xml version="1.0"?>
<package format="3">
  <name>two_dof_arm</name>
  <version>0.0.0</version>
  <description>TODO: Package description goes here</description>
  <maintainer email="rola@mmmi.sdu.dk">rola</maintainer>
  <license>TODO: License declaration</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

Building the URDF prototype

- /urdf/two_dof_arm.urdf
- Primitives are your friend
- Minimal setup
- Making sure the controllers work
- Normally not done manually…but often unavoidable to tune things.



Base structure: two_dof_arm.urdf

```
<?xml version="1.0"?>
<robot name="two_dof_arm">

</robot>
```

```
<?xml version="1.0"?>
<robot name="two_dof_arm">
        <link name="base_link">

        </link>
</robot>
```

```
<?xml version="1.0"?>
<robot name="two_dof_arm">
        <link name="base_link">
                <inertial>
                </inertial>

                <visual
                </visual>

                <collision
                </collision>
        </link>
</robot>
```

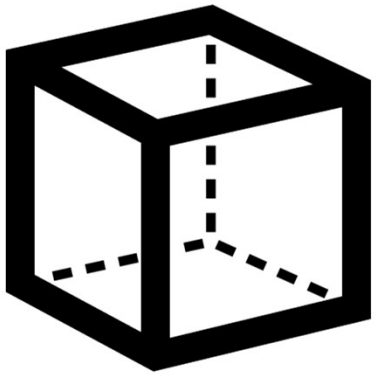
```
<inertial>
  <origin xyz="0 0 0" rpy="0 0 0"/>
  <mass value="1.0"/>
  <inertia ixx="0.01" ixy="0.0" ixz="0.0"
           iyy="0.01" iyz="0.0"
           izz="0.01"/>
</inertial>
```

```
<visual>
  <origin xyz="0 0 0.1" rpy="0 0 0"/>
  <geometry>
    <box size="0.2 0.2 0.2"/>
  </geometry>
</visual>
```

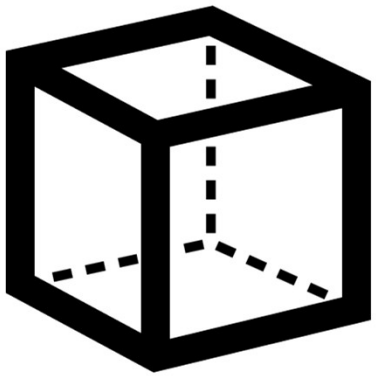
```
<visual>
  <origin xyz="0 0 0.1" rpy="0 0 0"/>
  <geometry>
    <box size="0.2 0.2 0.2"/>
  </geometry>
</visual>
```

```
<collision>
  <origin xyz="0 0 0.1" rpy="0 0 0"/>
  <geometry>
    <box size="0.2 0.2 0.2"/>
  </geometry>
</collision>
```

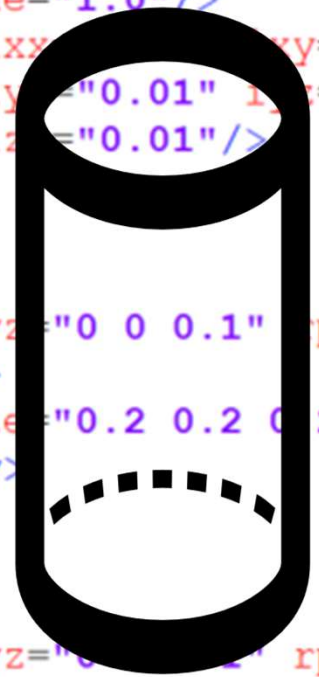
```xml
<!-- Base link -->
<link name="base_link">
  <inertial>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <mass value="1.0"/>
    <inertia ixx="0.01" ixy="0.0" ixz="0.0"
             iyy="0.01" iyz="0.0"
             izz="0.01"/>
  </inertial>

  <visual>
    <origin xyz="0 0 0.1" rpy="0 0 0"/>
    <geometry>
      <box size="0.2 0.2 0.2"/>
    </geometry>
  </visual>

  <collision>
    <origin xyz="0 0 0.1" rpy="0 0 0"/>
    <geometry>
      <box size="0.2 0.2 0.2"/>
    </geometry>
  </collision>
</link>
```

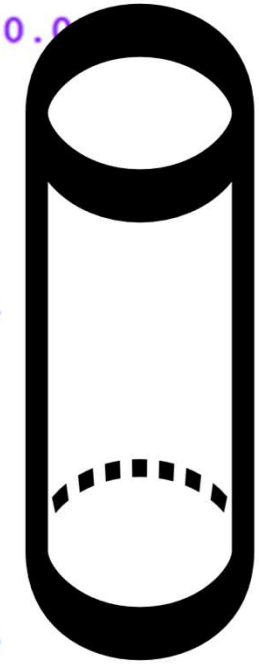


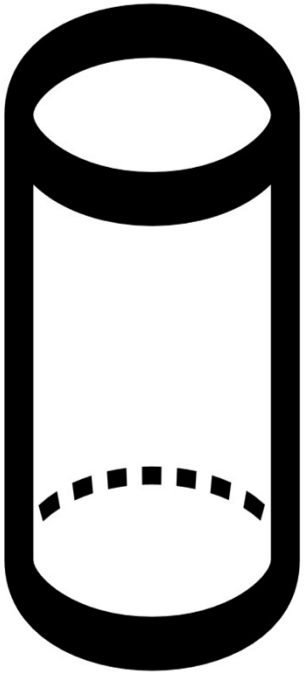
```xml
<!-- Base link -->
<link name="base_link">
  <inertial>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <mass value="1.0"/>
    <inertia ixx="0.01" ixy="0.0" ixz="0.0"
             iyy="0.01" iyz="0.0"
             izz="0.01"/>
  </inertial>

  <visual>
    <origin xyz="0 0 0.1" rpy="0 0 0"/>
    <geometry>
      <box size="0.2 0.2 0.2"/>
    </geometry>
  </visual>

  <collision>
    <origin xyz="0 0 0.1" rpy="0 0 0"/>
    <geometry>
      <box size="0.2 0.2 0.2"/>
    </geometry>
  </collision>
</link>
```



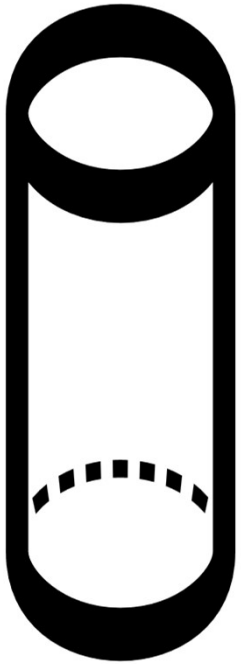
```
<!-- Base link -->
<link name="base_link">
  <inertial>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <mass value="1.0"/>
    <inertia ixx[illegible]y="0.0" ixz="0.0
             iy[illegible]="0.01" i[illegible]="0.0"
             iz[illegible]="0.01"/>
  </inertial>

  <visual>
    <origin xyz[illegible]"0 0 0.1" [illegible]py="0 0 0"/>
    <geometry>
      <box size[illegible]"0.2 0.2 [illegible]2"/>
    </geometry>
  </visual>

  <collision>
    <origin xyz="[illegible] rpy="0 0 0"/>
    <geometry>
      <box size="0.2 0.2 0.2"/>
    </geometry>
  </collision>
</link>
```







```
<!-- Link 1 -->
<link name="link1">
  <inertial>
    <origin xyz="0.0 0 0.15" rpy="0 0 0"/>
    <mass value="0.5"/>
    <inertia ixx="0.005" ixy="0.0" ixz="0.0"
             iyy="0.005" iyz="0.0"
             izz="0.005"/>
  </inertial>

  <visual>
    <origin xyz="0.0 0 0.15" rpy="0 0 0"/>
    <geometry>
      <cylinder radius="0.03" length="0.3"/>
    </geometry>
  </visual>

  <collision>
    <origin xyz="0.0 0 0.15" rpy="0 0 0"/>
    <geometry>
      <cylinder radius="0.03" length="0.3"/>
    </geometry>
  </collision>
</link>
```



```
<?xml version="1.0"?>
<robot name="two_dof_arm">
        <link name="base_link">
                <inertial>
                </inertial>

                <visual
                </visual>

                <collision
                </collision>
        </link>
        <joint>

        </joint>
        <link> [...]
</robot>
```

```
<joint name="joint1" type="revolute">
  <parent link="base_link"/>
  <child link="link1"/>
  <origin xyz="0 0 0.2" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
  <limit lower="-3.14" upper="3.14" effort="10" velocity="3"/>
</joint>
```

```
<?xml version="1.0"?>
<robot name="two_dof_arm">
        <link name="base_link">
                <inertial></inertial>
                <visual</visual>
                <collision</collision>
        </link>
        <joint></joint>
        <link> [...] </link>
        [...]
         <ros2_control> </ros2_control>
        <gazebo> </gazebo>
</robot>
```

```xml
<ros2_control name="GazeboSystem" type="system">
  <hardware>
    <plugin>gazebo_ros2_control/GazeboSystem</plugin>
  </hardware>

  <joint name="joint1">
    <command_interface name="position"/>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
  </joint>

  <joint name="joint2">
    <command_interface name="position"/>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
  </joint>
</ros2_control>
```

```xml
<gazebo>
  <plugin name="gazebo_ros2_control" filename="libgazebo_ros2_control.so">
    <parameters>
      /home/robert/MAS_ros2_workspace/install/two_dof_arm/share/two_dof_arm/config/controllers.yaml
    </parameters>
  </plugin>
</gazebo>
```

Controller of the URDF prototype

- /config/controllers.yaml

```yaml
controller_manager:
  ros__parameters:
    update_rate: 100

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    arm_controller:
      type: forward_command_controller/ForwardCommandController

arm_controller:
  ros__parameters:
    joints: [joint1, joint2]
    interface_name: position
```

Alternative controller (to forward_command_controller/ForwardCommandController)

joint_trajectory_controller/JointTrajectoryController

position_controllers/JointGroupPositionController

velocity_controllers/JointGroupVelocityController

effort_controllers/JointGroupEffortController

forward_command_controller/ForwardCommandController

diff_drive_controller/DiffDriveController

gripper_controllers/GripperCommandController

Controller of the URDF prototype

- /config/controllers.yaml

```yaml
controller_manager:
  ros__parameters:
    update_rate: 100

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    arm_controller:
      type: forward_command_controller/ForwardCommandController

arm_controller:
  ros__parameters:
    joints: [joint1, joint2]
    interface_name: position
```

Nearly there: sim.launch.py

- launch/ sim.launch.py

```python
import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node


def generate_launch_description():

    pkg_share = get_package_share_directory("two_dof_arm")
    urdf_file = os.path.join(pkg_share, "urdf", "two_dof_arm.urdf")
    controllers_file = os.path.join(pkg_share, "config", "controllers.yaml")
```

```python
# Gazebo (classic)
gazebo = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        os.path.join(
            get_package_share_directory("gazebo_ros"),
            "launch",
            "gazebo.launch.py",
        )
    )
)
```

```
# Robot State Publisher (TF only)
robot_state_publisher = Node(
    package="robot_state_publisher",
    executable="robot_state_publisher",
    parameters=[{"robot_description": open(urdf_file).read()}],
    output="screen",
)
```

```python
# Spawn robot directly from URDF file (NOT from topic)
spawn_entity = Node(
    package="gazebo_ros",
    executable="spawn_entity.py",
    arguments=[
        "-entity", "two_dof_arm",
        "-file", urdf_file,
    ],
    output="screen",
)
```

```
# Joint state broadcaster
joint_state_broadcaster = Node(
    package="controller_manager",
    executable="spawner",
    arguments=[
        "joint_state_broadcaster",
        "--controller-manager", "/controller_manager",
    ],
    output="screen",
)
```

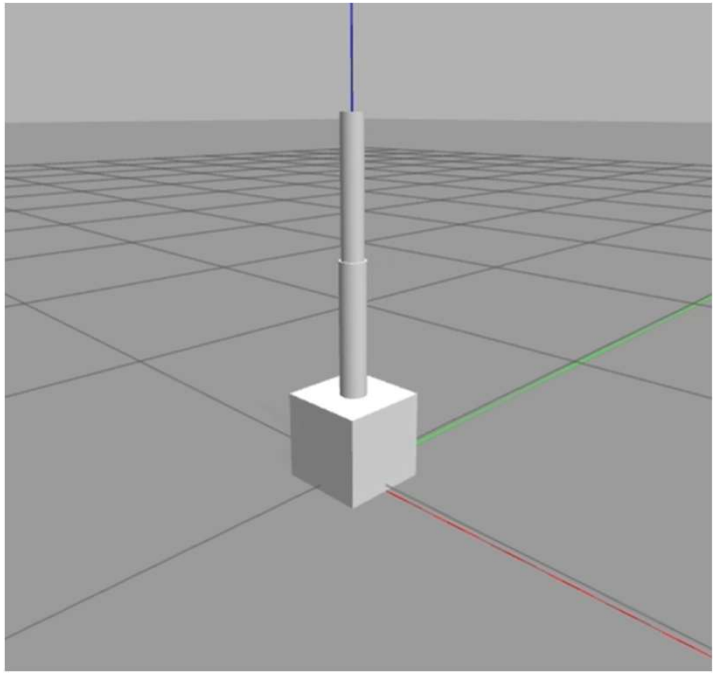
```
# Arm controller
arm_controller = Node(
    package="controller_manager",
    executable="spawner",
    arguments=[
        "arm_controller",
        "--controller-manager", "/controller_manager",
        "--param-file", controllers_file,
    ],
    output="screen",
)
```

```
return LaunchDescription([
    gazebo,
    robot_state_publisher,
    spawn_entity,
    joint_state_broadcaster,
    arm_controller,
])
```

Finally

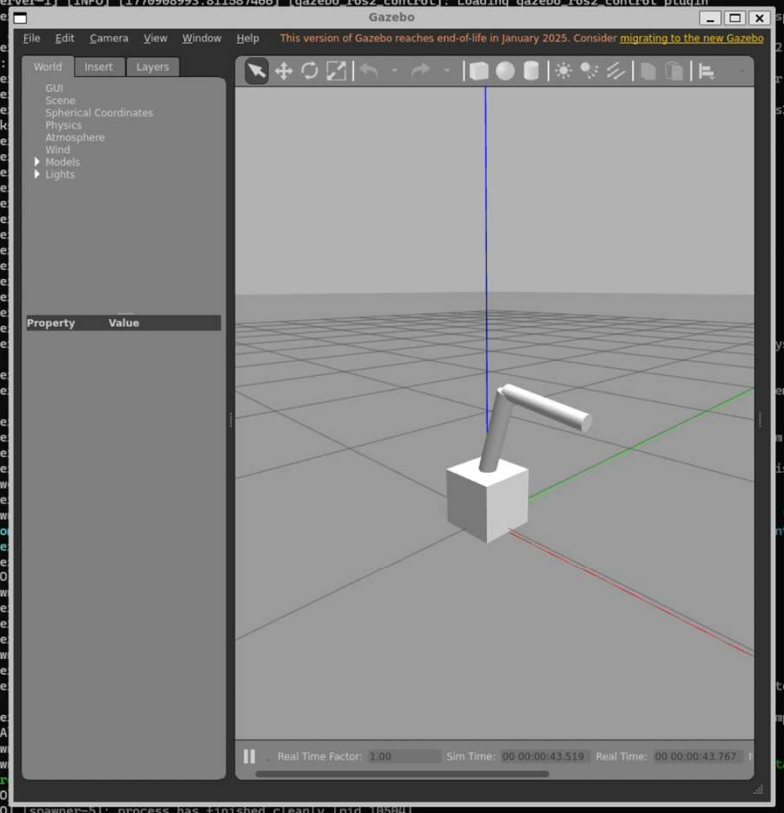
- ros2 launch two_dof_arm sim.launch.py

- ros2 topic pub --once /arm_controller/commands std_msgs/msg/Float64MultiArray "{data: [0, 1.4]}"



robert@ROLA1: ~/MAS_ros2_

```
[robot_state_publisher-3] [INFO] [1770908989.274104495] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1770908989.274177119] [robot_state_publisher]: got segment link1
[robot_state_publisher-3] [INFO] [1770908989.274184495] [robot_state_publisher]: got segment link2
[spawner-6] [INFO] [1770908989.646618380] [spawner_arm_controller]: waiting for service /controller_manager/lis
t_controllers to become available...
[spawner-5] [INFO] [1770908989.676826544] [spawner_joint_state_broadcaster]: waiting for service /controller_ma
nager/list_controllers to become available...
[spawn_entity.py-4] [INFO] [1770908989.829924588] [spawn_entity]: Spawn Entity started
[spawn_entity.py-4] [INFO] [1770908989.830265031] [spawn_entity]: Loading entity XML from file /home/robert/MAS
_ros2_workspace/install/two_dof_arm/share/two_dof_arm/urdf/two_dof_arm.urdf
[spawn_entity.py-4] [INFO] [1770908989.833847762] [spawn_entity]: Waiting for service /spawn_entity, timeout =
30
[spawn_entity.py-4] [INFO] [1770908989.834078493] [spawn_entity]: Waiting for service /spawn_entity
[spawn_entity.py-4] [INFO] [1770908993.610281692] [spawn_entity]: Calling service /spawn_entity
[spawn_entity.py-4] [INFO] [1770908993.781929918] [spawn_entity]: Spawn status: SpawnEntity: Successfully spawn
ed entity [two_dof_arm]
[gzserver-1] [INFO] [1770908993.811587466] [gazebo_ros2_control]: Loading gazebo_ros2_control plugin
[INFO] [spawner-5]: process has finished cleanly [pid 10504]
```



robert@ROLA1: ~/MAS_ros2_

```
robert@ROLA1:~/MAS_ros2_workspace/src/apps$ ros2 topic pub --once /arm_controller/commands std_msgs/msg/Float64MultiArray "{data: [0, 1.4]}"
publisher: beginning loop
publishing #1: std_msgs.msg.Float64MultiArray(layout=std_msgs.msg.MultiArrayLayout(dim=[], data_offset=0), data=[0.0, 1.4])

robert@ROLA1:~/MAS_ros2_workspace/src/apps$ ros2 topic pub --once /arm_controller/commands std_msgs/msg/Float64MultiArray "{data: [0.3, 1.4]}"
publisher: beginning loop
publishing #1: std_msgs.msg.Float64MultiArray(layout=std_msgs.msg.MultiArrayLayout(dim=[], data_offset=0), data=[0.3, 1.4])

robert@ROLA1:~/MAS_ros2_workspace/src/apps$
```